

MODELAÇÃO E AUTOMAÇÃO DE PROCESSOS INDUSTRIAIS

MEEC

Laboratório 2 – Sistema de alarme para deteção de intrusão

Autores:

Margarida Canas (106080)

Beatriz Coelho (107113)

Tiago Gomes (106611)

margarida.canas@tecnico.ulisboa.pt

beatriz.h.coelho@tecnico.ulisboa.pt

tiago.ramos.gomes@tecnico.ulisboa.pt

GrupoA1

2025/2026 – 2º Semestre, P4

Índice

Introdução	2
Parte A - Implementação de um alarme através de switches	3
A1 - Diagrama do Hardware do PLC e do Terminal	3
A2 - Identificação dos I/O do painel de alarme	3
A3 - Programa de teste do hardware	4
A4 - Programa de acesso ao Buzzer a partir do painel de alarme	5
A5 - Demonstração do programa no painel de alarme	7
Section: Main	7
Subrotina: SR_alarm_off	8
Subrotina: SR_alarm_presence	8
Subrotina: SR_alarm_active	9
A6 - Time plot do funcionamento do alarme	10
Parte B - Detecção e identificação de teclas de um teclado matricial	13
B1 - Identificação de IO do Teclado	13
B2 - Estratégia de Leitura e Identificação	13
Sub-rotina <i>SR_get_key</i>	13
Sub-rotina <i>SR_get_multiple_keys</i>	15
B3 - Registo de Dados	15
Limite do Buffer e Funcionalidade de <i>Reset</i>	16
Parte C - Integração da leitura do teclado e dos modos de alarme	19
C2 - Programa para integração no PLC	20
C3 - Demonstração do funcionamento	21
C4 - Tempo mínimo para desativar o alarme	23
C5 - Strings logging	24
C6 - Modificações ao data logger para gerar plots em tempo real dos sinais do PLC	25
C7 - Possíveis melhorias	26
Conclusões	26

Introdução

O presente relatório descreve o trabalho experimental realizado no âmbito da unidade curricular de Modelação e Automação de Processos Industriais. O objetivo central deste laboratório é a implementação de um sistema de alarme para deteção de intrusão, recorrendo a um Controlador Lógico Programável (PLC) Schneider da gama Premium, modelo TSX P57, e ao ambiente de desenvolvimento Unity Pro.

A componente experimental assenta na interface entre o PLC e uma consola de alarme, recorrendo a diversos materiais:

- Interface de I/O: entradas e saídas digitais do PLC, associadas aos sensores, LEDs e buzzer da consola.
- Componentes de input: switch de presença, switch de alarme, switch de janela e teclado matricial de 12 teclas.
- Componentes de output: LEDs verde, amarelo e vermelho, e buzzer.
- Hardware simulado: terminal virtual desenvolvido em Matlab, com comunicação Modbus com o PLC.

Através desta infraestrutura, o trabalho divide-se na implementação do funcionamento base do alarme através de switches, LEDs e buzzer (Parte A), na leitura e identificação de teclas de um teclado matricial 4x3 (Parte B) e na integração global do sistema de alarme, incluindo a seleção dos modos de funcionamento e a validação de sequências introduzidas pelo teclado (Parte C).

Parte A - Implementação de um alarme através de switches

A1 - Diagrama do Hardware do PLC e do Terminal

De acordo com o enunciado desta questão, foi-nos pedido que completássemos com as respectivas ligações e endereços o diagrama de hardware real e simulado. É apresentado na figura 1 o diagrama completo.

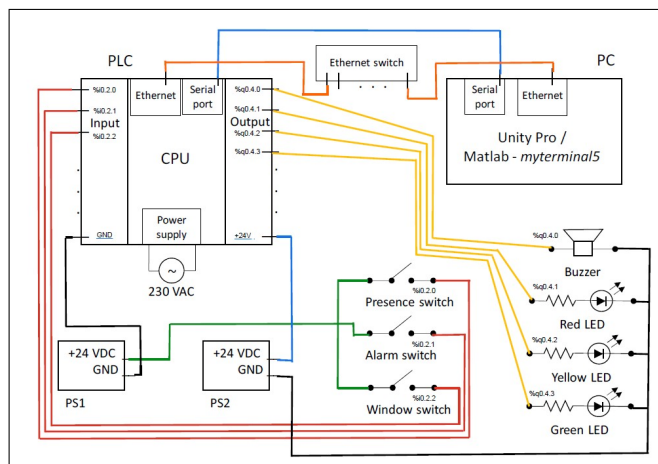


Figura 1: Diagrama do hardware real e simulado completo.

A2 - Identificação dos I/O do painel de alarme

Na tabela que apresentamos abaixo, tabela 1, estão apresentados os endereços e os nomes dos identificadores da PLC que serão utilizados para desenvolver os programas que se apresentam nas próximas alíneas. É importante notar que, de acordo com a sugestão dos professores no enunciado deste projeto, foi atribuída a convenção de nomes (I, O, M, TM, SR), às variáveis utilizadas nos programas.

Nome	Endereço	Endereço de memória	Tipo
I_presence	%i0.2.0	%m20	input
I_alarm	%i0.2.1	%m21	input
I_window	%i0.2.2	%m22	input
O_buzzer	%q0.4.0	%m40	output
O_red	%q0.4.1	%m41	output
O_yellow	%q0.4.2	%m42	output
O_green	%q0.4.3	%m43	output

Tabela 1: Tabela de endereços dos identificadores da PLC.

A3 - Programa de teste do hardware

Nesta secção foi-nos pedido que criássemos um programa em **Structured Text (ST)** que ativasse cada um dos 4 outputs do módulo de alarme (buzzer, LED verde, LED vermelho e LED amarelo), durante 0.5s alternadamente.

De modo a implementarmos este programa, recorremos a um temporizador TON com duração de 0.5s.

```
(* square wave 0.5s *)  
my_timer1 (IN := (NOT my_timer1.Q), PT := t#0.5s);
```

Figura 2: Timer TON com duração de 0.5s.

Conforme podemos observar na figura 2 o timer, foi criado com um timing predefinido de 0.5 segundos (t#0.5s). A sua entrada corresponde à negação da sua própria saída. Deste modo, assim que o timer contabiliza 0.5 segundos, a saída Q assume o valor de 1 durante exatamente um scan-cycle do PLC. No ciclo seguinte, o input IN torna-se então 0 reiniciando o timer. Cria-se assim um *clock* que dispara continuamente a cada 0.5s.

Adicionalmente, criou-se uma variável **counter** incrementada a cada rising edge (RE) de (my_timer1.Q), isto é, a cada 0.5s. Esta assume um valor entre 0 e 3, reiniciando depois o ciclo de contagem. Deste modo, o valor acumulado nesta variável de contagem é utilizado através de uma lógica de IF ... ELSIF para ativar, um de cada vez, os 4 outputs.

Concluída assim a implementação do programa cíclico de 0.5s para o teste de todos os outputs do módulo de alarme, foi produzido um gráfico de data logging com recurso a um script de **Matlab**. O gráfico resultante é apresentado abaixo, figura 3, e ilustrando a evolução temporal de cada um dos 4 outputs, revelando a sua ativação alternada durante períodos de 0.5 segundos.

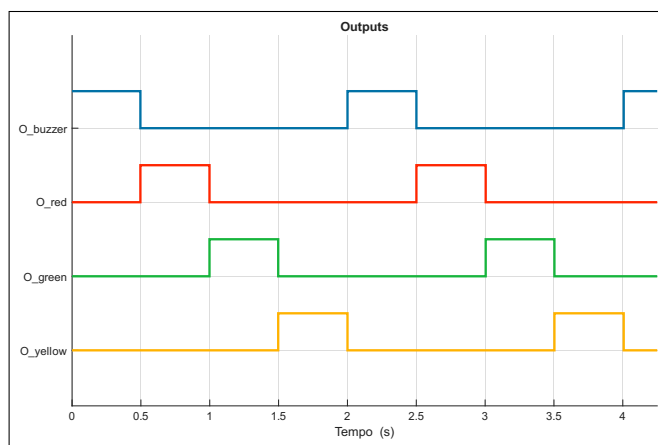


Figura 3: Evolução temporal dos Outputs do módulo de alarme.

A4 - Programa de acesso ao Buzzer a partir do painel de alarme

Nesta alínea foi-nos pedido que criássemos um programa, novamente em **Structured Text (ST)**, que testasse os múltiplos acessos ao *buzzer*, do painel de alarme.

Dentro deste programa, deverão existir 4 maneiras de aceder ao buzzer. Duas variáveis diferentes de memória ativarão o buzzer (ON). Uma terceira servirá para desligar incondicionalmente o buzzer, enquanto uma quarta iniciará um ciclo periódico com o buzzer a alternar entre ON durante 1s e OFF durante 2s. No caso de tanto o modo periódico estar ativo como um dos 2 modos que colocam o buzzer a ON, o primeiro terá precedência sobre os segundos.

De modo a implementar esta lógica, criamos 4 variáveis de memória que utilizamos para definir o estado em que se encontrava o programa. São elas `M_buzzer_off` que desliga o buzzer incondicionalmente, `M_buzzer_altern` que inicia o modo alternado e `M_buzzer_on1` e `M_buzzer_on2` que colocam o buzzer ON. De modo a implementar o sistema de prioridades do programa utilizou-se a estrutura que se apresenta na figura 4.

```
(* unconditionally turned off by another memory variable*)
IF M_buzzer_off THEN

    O_buzzer := 0;

(* final way with another mem var, buzzer is 1s ON and then 2s OFF*)
ELSIF (M_buzzer_altern) THEN

    O_buzzer := buzzer_pulse;

(* turned on by two alternative memory variable*)
ELSIF (M_buzzer_on1 or M_buzzer_on2) THEN

    O_buzzer := 1;

ELSE

    O_buzzer := 0;

END_IF;
```

Figura 4: Estrutura IF ELSIF utilizada em A4.

Observando atentamente a figura acima, verificamos que a primeira condição da estrutura IF ... ELSIF passa por colocar o buzzer (`O_buzzer`) a OFF, caso se esteja no modo `M_buzzer_off`. Como na estrutura IF ... ELSIF apenas se executada a primeira condição verificada, garantimos que o buzzer fica OFF incondicionalmente, independentemente dos outros Modos estarem também ativos ou não.

Para auxiliar o modo alternado foi criado um timer TON (figura 5) com uma temporização de 3s. Este timer é alimentado pela negação da sua saída (funcionando como um clock cíclico) e pela variável `M_buzzer_altern`, garantindo que o timer apenas está ativo quando estamos no modo alternado. De seguida, verifica-se se o tempo decorrido é menor ou igual a 1s, igualando

esta condição à variável `buzzer_pulse`.

```
timer_3s(IN := (M_buzzer_altern AND NOT timer_3s.Q), PT := t#3s);
buzzer_pulse := (timer_3s.ET <= t#1s);
```

Figura 5: Timer TON com duração de 3s.

Analizando novamente a estrutura IF ... ELSIF (figura 4), observamos que a segunda condição da nossa estrutura é então a condição que verifica `M_buzzer_altern`, e onde se iguala o buzzer (`O_buzzer`) à variável `buzzer_pulse`. Como tal, cria-se então o efeito alternado desejado. O buzzer, ciclicamente, estará ON durante 1s (`buzzer_pulse = 1`) e OFF durante outros dois, recomeçando depois novamente este ciclo.

É apenas em último lugar que verificamos `M_buzzer_on1` e `M_buzzer_on2` que se limitam a colocar o buzzer ON. Deste modo, como estas duas variáveis são verificadas após `M_buzzer_altern` garantimos que, no caso do comando ser simultaneamente `M_buzzer_altern` e buzzer ON, o primeiro tem precedência sobre qualquer um dos dois modos que originam o segundo.

Em contexto laboratorial, para podermos testar o programa desenvolvido, utilizamos os inputs do painel de alarme para controlar os modos do programa. Foi utilizado o switch de três posições para os dois modos de buzzer ON (`M_buzzer_on1 := %I0.2.0` e `M_buzzer_on2 := %I0.2.1`), o interruptor associado à janela para o modo OFF (`M_buzzer_off := NOT %I0.2.2`), e o modo alternado `M_buzzer_altern` foi controlado em *runtime* diretamente na *Animation Table*, devido à falta de um 3º interruptor.

Concluída a implementação produziu-se um gráfico de data logging com recurso a um script de Matlab. O gráfico resultante é apresentado abaixo, figura 6, ilustrando a evolução temporal de cada um dos 4 modos e do buzzer.

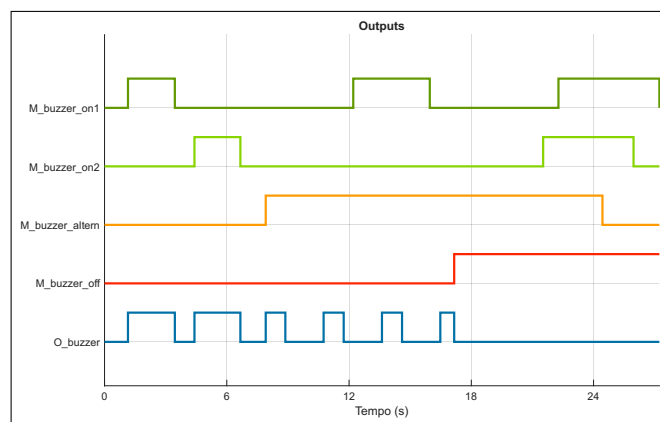


Figura 6: Evolução temporal das 4 variáveis de modo e do buzzer.

Analisando a figura é relevante verificar que o buzzer está OFF independentemente dos outros modos, quando `M.buzzer_off` está ativo (a partir de $t = 17s$). Adicionalmente entre $t = 8s$ e $t = 17s$ quando nos encontramos tanto no modo alternado ON/OFF como no modo ON, o buzzer permanece no modo alternado.

A5 - Demonstração do programa no painel de alarme

Na secção A5, é pedido que desenvolvamos um programa que implemente apenas as características genéricas do alarme. Este programa deve conter 3 modos, **OFF** (**mode 1**), **Presence Detection** (**mode 2**) e **Active** (**mode 3**). Cada um dos modos deve ser implementado de acordo com as indicações que se encontram no Anexo A do guia laboratorial fornecido.

O programa foi estruturado numa secção **Main** e em três subrotinas (SR) diferentes, **SR_alarm_off**, **SR_alarm_presence** e **SR_alarm_active**.

Section: Main

Em primeiro lugar é preciso salientar que devido à existência de apenas dois interruptores, consideramos o interruptor de três posições como aquele que selecionaria o modo de operação do alarme (`I_alarm = %i0.2.1` para o Modo 3, `I_presence = %i0.2.0` para o Modo 2 e nenhum dos dois inputs para o Modo 1) enquanto que o switch `I_window = %i0.2.2` é o responsável por funcionar como sensor de movimento.

Na secção **Main**, começamos verificar se a tecla `#key` está ativa, alimentado o output `%q0.4.6` e o igualando `I_key` ao input `%q0.4.6`. No scan-cycle onde `#key` for pressionada, `I_key` ficará temporariamente a 1. Como o objetivo é desligar o alarme com esta tecla (isto é, retornar ao Modo 1), pressionar a tecla `#key` faz o modo do alarme mudar para 1. Contudo, como o switch físico continua a indicar `I_alarm = 1`, retirar o dedo da tecla `I_key` faz-nos continuar no modo 3. Assim para retornar ao modo 1, é necessário retornar o switch de três posições à posição neutra (`%i0.2.0 = 0` e `%i0.2.1 = 0`).

Deste modo, é criada uma variável de memória `M_key_pressed` que fica a 1 quando (`I_key = 1`). Caso esta variável esteja a 0, a correspondência entre as posições dos switches e o modo correspondente é feita da maneira anteriormente explicada conforme o código presente na imagem [7](#)


```

IF (M_key_pressed = 0) THEN
  IF (I_alarm = 1) THEN
    M_MODE := 3;
  ELSIF (I_presence = 1) THEN
    M_MODE := 2;
  ELSE
    M_MODE := 1;
  END_IF;
ELSE
  M_MODE := 1;
  IF (RE(I_alarm) OR FE(I_alarm) OR RE(I_presence) OR FE(I_presence)) THEN
    M_key_pressed := 0;
  END_IF;
END_IF;

```

Figura 7: Correspondência entre M_MODE e as posições dos switches conforme o valor de M_key_pressed.

Por outro lado, verificamos também na imagem 7, que caso `M_key_pressed = 1` é forçado sempre o modo de funcionamento 1 (`M_MODE = 1`). Deste modo, para poder desativar a variável `M_key_pressed` (e consequentemente sair do modo de funcionamento 1), é preciso forçar o retorno do switch à posição neutra (detetar um Rising (RE) ou Falling Edge (FE) quer de `I_alarm` ou de `I_presence`).

Deste modo, garantimos que quando o alarme está ativo (Modo 3), é desativado quando a tecla #key é pressionada (ficando no Modo 1). O Modo do alarme permanecerá no 1 até que o switch de três posições retorne à posição neutra (que codifica o Modo 1), altura na qual a variável `M_key_pressed` será desativada permitindo ao alarme, variar novamente entre os 3 modos disponíveis.

A secção main termina com uma estrutura IF/ELSE onde é realizada a correspondência entre o modo de funcionamento do alarme `M_MODE` e a subrotina correspondente a ser chamada. As três subrotinas que mencionámos anteriormente serão agora explicadas.

Subrotina: SR_alarm_off

Esta subrotina implementa o Mode 1 - OFF, correspondente ao alarme desativado. Deste modo, nesta subrotina todos os outputs do painel de alarme `0_buzzer`, `0_red`, `0_green` e `0_yellow` são colocados a 0.

Subrotina: SR_alarm_presence

Esta subrotina implementa o Mode 2 - Presence Decection. Nesta rotina, começamos por implementar um timer TP (figura 8) com duração de 5 segundos e que é ativado aquando da deteção de algum movimento (`RE(I_window)`).

```

(* criar um timer de 5 segundos para a luz*)
timer_presence (IN := RE(I_window), PT := t#5s);
O_green := timer_presence.Q;|
(* enquanto o timer for menos que um segundo dispara o buzzer*)
O_buzzer := timer_presence.Q AND timer_presence.ET <= t#1s ;

```

Figura 8: Timer TP de 5s implementado em SR_alarm_presence.

Conforme se observa na figura 8, a saída do timer `timer_presence.Q`, alimenta o LED verde `O_green`, garantindo assim que um dos LEDs fica ativo durante 5s aquando da deteção de movimento. Adicionalmente, verifica-se se o tempo decorrido desde o disparo do timer é inferior a 1s (`timer_presence.ET`), situação na qual se ativará o buzzer. Deste modo, garante-se também que aquando da deteção de movimento e paralelamente ao LED de duração 5s, o buzzer emita um sinal acústico com duração de 1s.

Subrotina: SR_alarm_active

Esta subrotina implementa o Mode 3 - Active. Nesta rotina começamos por implementar um timer TON de 30s, que é ativado mal entramos na subrotina. Durante os 30s iniciais durante os quais `t_wait30s = 0`, o LED amarelo (`O_yellow`) está continuamente ligado, sinalizado ao utilizador que deve abandonar o espaço.

De seguida, após o final dos 30s e com (`t_wait30s.Q = 1`) é ativado o LED verde (`O_green`), sinalizando que o alarme está ativo. Com o alarme ativo, e caso seja detectado algum movimento é ativada a variável de memória `M_intruder` e o alarme evolui para uma fase de warning.

Nesta fase de warning, são iniciados dois timers TON, como se pode visualizar na figura 9, abaixo:

```

(* timer para o buzzer disparar 5s *)
t_buzzer(IN := M_intruder, PT := t#5s); (*TON*)|
(* timer warning sequence 3s *)
t_warning(IN := (M_intruder AND NOT t_warning.Q), PT := t#3s); (*TON*)

```

Figura 9: Timers TON utilizados em SR_alarm_active.

O timer `t_buzzer` é iniciado quando se deteta algum movimento `M_intruder = 1`, com uma duração de 5 segundos. No final deste período do tempo a sua saída `t_buzzer.Q` terá o valor de 1. A variável `O_buzzer` é igualada no programa à saída deste temporizador, o que fará com que o buzzer fique ON 5 segundos após a deteção de um intruso com o alarme ativo e assim continue até ser desativado.

Do mesmo modo, a entrada do timer `t_warning` é simultaneamente a variável que deteta algum movimento, e a negação da sua própria saída `t_warning.Q`, conferindo a este temporizador um carácter cíclico. O timer em questão é iniciado com uma duração de 3 segundos (`t#3s`). O LED vermelho (`O_red`) é assim ativado caso se detete a presença de um intruso e o tempo desde o

início deste timer seja inferior a um segundo ($t_{\text{warning.ET}} \leq t_{\#1s}$). Deste modo, aquando da deteção de movimento, o LED vermelho comporta-se como um sinal periódico até à sua desativação, ficando 1 segundo ON e 2 segundos OFF a cada ciclo.

Os gráficos realizados com recurso ao data logging que exemplificam o funcionamento da secção aqui apresentada apresentam-se na secção **A6**.

A6 - Time plot do funcionamento do alarme

Nesta alínea, é-nos pedido que apresentemos gráficos de utilizando data logging para ilustrar o funcionamento do alarme, isto é, do código desenvolvido na secção anterior A5. Concluída a implementação do código de A5, produziram-se então gráficos de data logging com recurso a um script de Matlab. Devido à elevada duração da experiência os Modos 2 e 3 de funcionamento do alarme, são retratados em gráficos distintos.

Para ilustrar o funcionamento do modo 2 do alarme, são apresentados 2 gráficos, as Figuras 10 e 11.

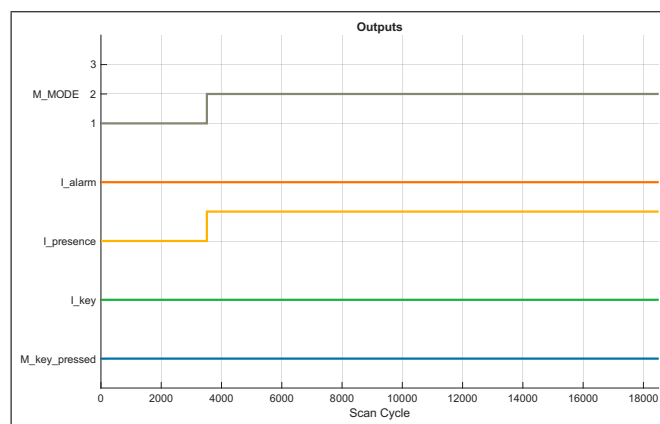


Figura 10: Evolução temporal dos inputs que codificam o modo de funcionamento do alarme.

O gráfico 10 ilustra a relação da variável `M_MODE` com os inputs do switch de 3 posições. Ao analisar a figura podemos perceber que conforme referido anteriormente é o input `I_presence = %i0.2.0` que codifica o modo 2 - Presence Detection.

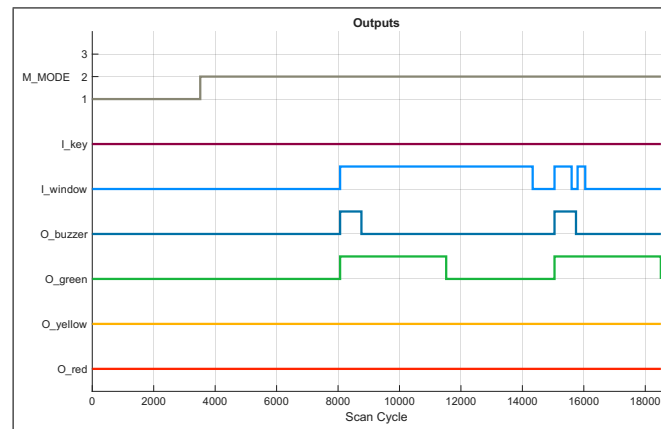


Figura 11: Evolução temporal das variáveis relacionadas ao Modo 2.

No figura 11 está ilustrado então o funcionamento do modo 2. Após a ativação pelo utilizador do modo 2, nada acontece imediatamente. É apenas aquando da deteção de uma presença/intrusão, ilustrado pela variável `I_window`, que se produz um sinal sonoro com duração de 1 segundo (`O_buzzer`). Concorrentemente com o sinal acústico, é também no momento da deteção de uma presença que uma das lâmpadas do módulo de alarme, o LED verde, irá ficar ativa durante um periodo de 5 segundos.

É relevante notar que é apenas o início de deteção do movimento que gera este tipo de resposta. Para este ciclo ser novamente iniciado de novo é necessário que se deixe de verificar a presença de movimento (`I_window` = 0) e posteriormente seja detetada novamente (aproximadamente no scan cycle 15000) para se iniciar este ciclo de deteção que conta com o LED verde e o buzzer.

Para ilustrar o funcionamento do modo 3 do alarme, são apresentados 2 gráficos, as Figuras 12 e 13.

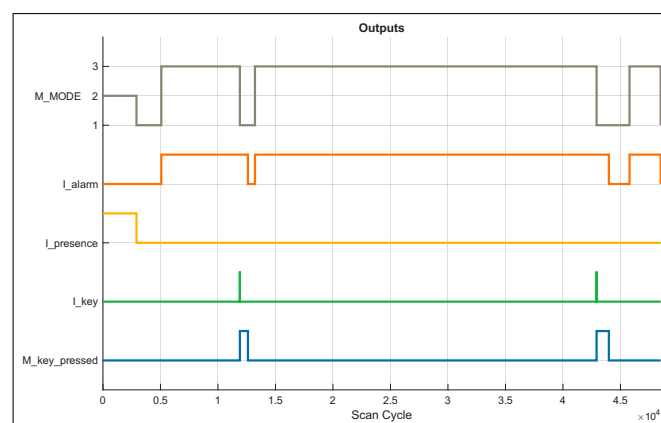


Figura 12: Evolução temporal dos inputs que codificam o modo de funcionamento do alarme.

O gráfico 12 ilustra a relação da variável `M_MODE` com os inputs do switch de 3 posições. Ao analisar a figura podemos perceber que conforme referido anteriormente é o input `I_alarm` =

%i0.2.1 que codificam o modo 3. É relevante também apontar que estando o alarme no modo de funcionamento 3, no momento em que é pressionada a tecla #key, conseguimos visualizar o impulso em I_key. É relevante apontar que neste instante M_MODE retorna imediatamente ao Modo 1 e a variável M_key_pressed é automaticamente ativa. A transição para outros modos de funcionamento para a ser apenas possível no instante em que M_key_pressed retorna a 0, momento esse que corresponde ao retorno do switch à posição neutra que codifica o modo 1 (momento em que I_presence = %i0.2.0 corresponde a 0, e o switch do modo 3 I_alarm = %i0.2.1 retorna a 0).

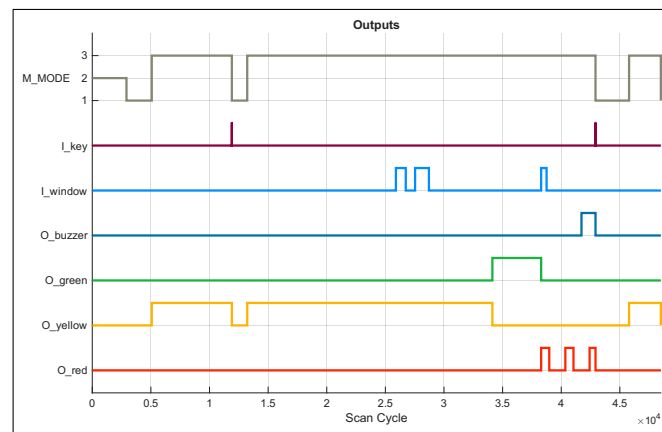


Figura 13: Evolução temporal das variáveis relacionadas ao Modo 2.

No figura 13 está ilustrado então o funcionamento do modo 3. Primeiramente, imediatamente aquando da ativação do Modo 3 ($M_MODE = 3$), o LED amarelo fica ativo continuamente, sinalizando os 30 segundos que o utilizador tem para sair do local. Esta fase na qual o alarme ainda não está totalmentte ativo é interrompida pelo pressionar da tecla #key, ilustrado pela variável I_key. Isto leva o modo de funcionamento do alarme a mudar para o modo 1.

De seguida, o utilizador muda novamente M_MODE para 3 e novamente o LED amarelo fica ativo. Durante este periodo de 30 segundos de inatividade do alarme apesar da deteção de movimentos (I_window) nada acontece como seria expectável, visto ser o tempo que o utilizador se encontra a sair de casa e o alarme não se encontra propriamente ativo. Acabados estes 30 segundos apaga-se o LED amarelo e acende-se o LED verde (O_green) de forma contínua, sinalizando que o alarme se encontra activado.

Com o alarme ativo é detectado um movimento (I_window) o que leva ao desativar do LED verde e ao início do comportamento do LED vermelho (O_red), como um sinal periódico, alternado entre 1 segundo ON e 2 segundos OFF. Volvidos 5 segundos desde a deteção do intruso faz-se então ouvir de forma contínua o buzzer O_buzzer. Este comportamento do led vermelho e do buzzer é interrompido pelo pressionar da tecla #key, o que leva à transição do modo 3 para o modo de funcionamento 1, desativando todos os Ouputs do painel do alarme.

Parte B - Detecção e identificação de teclas de um teclado matricial

B1 - Identificação de IO do Teclado

Identificação das entradas e saídas do PLC, de e para o teclado, utilizadas no sistema de alarme de deteção de intrusão.

PLC Input (input address, %I)	PLC Identifier (variable name)
%i0.2.4	line_a
%i0.2.5	line_b
%i0.2.6	line_c
%i0.2.7	line_d

Tabela 2: Mapeamento de Entradas do Teclado (linhas)

PLC Output (output address, %Q)	PLC Identifier (variable name)
%q0.4.4	col_1
%q0.4.5	col_2
%q0.4.6	col_3

Tabela 3: Mapeamento de Saídas do Teclado (colunas)

B2 - Estratégia de Leitura e Identificação

A estratégia de leitura e identificação do teclado foi dividida em 2 sub-rotinas desenvolvidas em Structured Text (ST), como era pedido no enunciado, a *SR_get_key* para identificar teclas individuais e a *SR_get_multiple_keys* para armazenar as teclas detetadas no buffer e implementar a lógica de reset do buffer.

Sub-rotina *SR_get_key*

A sub-rotina *SR_get_key* implementa uma **inicialização de variáveis** que foi programada para ser executada apenas uma vez, no momento em que o programa arranca e a sub-rotina é chamada pela primeira vez.

Esta lógica é controlada através da instrução *IF NOT first_run_key THEN*. Como a variável booleana *first_run_key* inicia por defeito a FALSE na memória do PLC, o código dentro deste bloco é acionado apenas no primeiro ciclo de varrimento.

Dentro deste bloco inicial, o sistema define os estados de arranque de todas as variáveis cruciais para o controlo do teclado:

- Ativa o *reset_buffer* para garantir que o buffer de armazenamento das chaves começa completamente "limpo".
- Inicializa as variáveis *candidate_key* e *KEY* a -1 para indicar que nenhuma tecla está premida (*candidate_key*) ou que nenhuma tecla válida foi lida (*KEY*).
- Coloca o *counter*, variável que dita qual das três colunas do teclado deve ser energizada, a 0 para garantir que o varrimento começa pela primeira coluna.
- Coloca o contador que o programa utiliza para verificar que o utilizador retirou o dedo do teclado a 3, para indicar que não há nenhuma tecla primida em nenhuma das 3 colunas, *no_pressed_counter*.
- Desativa a flag *key_up*, que avisa a sub-rotina *SR_get_multiple_keys* que há uma tecla nova para ser processada, para garantir que começa desligada.
- Por fim, ativa a variável *first_run_key*, para impedir o programa de voltar a inicializar as variáveis nos ciclos seguintes.

A sub-rotina implementa um temporizador (*my_timer1*) para gerar um impulso periódico a cada 0.05 segundos. A cada impulso do temporizador, o programa incrementa um contador (*counter*) que toma os valores 0, 1 e 2. Este contador é utilizado num bloco *CASE* para energizar apenas uma das três colunas (*col_1*, *col_2*, *col_3*) de cada vez.

Enquanto uma coluna está energizada, o programa verifica o estado das entradas *line_a*, *line_b*, *line_c* e *line_d*. Se detetar exatamente uma linha ativa e se nas ultimas colunas lidas não tinha sido primida nenhuma tecla (*no_pressed_counter* $\neq$ 1), o cruzamento entre o valor do *counter* (coluna) e a linha ativa permite identificar a tecla premida e guardar esse valor na variável *candidate_key*.

Para evitar o registo errado de múltiplas teclas premidas em simultâneo na mesma coluna, o sistema conta o número de linhas ativas (*active_lines*). Se este valor for diferente de 1, a tecla candidata é rejeitada e forçada ao valor -1.

O programa garante que uma tecla não é lida repetidamente. Quando não existem linhas ativas (*active_lines* = 0), um contador (*no_pressed_counter*) é incrementado. Apenas quando este contador atinge um valor superior igual ou 3, ou seja, quando o programa volta a energizar a coluna onde a última tecla primida está e não deteta nenhuma tecla (o que confirma que a tecla foi fisicamente largada) é que o valor registado em *candidate_key* é validado para a variável final *KEY* e a flag *key_up* é ativada passando para *TRUE*.

Imediatamente após a validação de uma tecla, a saída *O_buzzer* é ativada provocando um feedback sonoro, e um segundo temporizador (*beep_timer*) garante que o som dura exatamente 0.05 segundos antes de ser desligado.

Sub-rotina *SR_get_multiple_keys*

Esta secção chama continuamente a sub-rotina *SR_get_key*.

Quando a flag *key_up* se encontra ativa e foi detetada uma tecla válida (*KEY* é diferente de -1), o programa verifica se há espaço num array designado *message*, ou seja, se o índice atual *msg_index* do array é menor que 10, o espaço total do array.

Se houver espaço, o valor da tecla é guardado em *message* na posição atual do índice *msg_index* e somamos 1 ao índice para avançar uma posição no array.

O valor da variável *current_key* é também atualizada para o valor da *KEY* para monitorização e a flag *key_up* é desativada.

Para além do armazenamento de teclas, esta secção também implementa o reset do array. O programa realiza a inicialização do array *message* através da flag *reset_buffer*.

Quando essa flag é ativada pelo programador, o array é preenchido com o valor -1 e as variáveis de índice (*msg_index*) e de tecla atual (*KEY* e *current_key*) são repostas a 0 e a -1, respetivamente.

B3 - Registo de Dados

De modo a monitorizar detalhadamente o funcionamento do programa durante a introdução do código "9963", foi necessário adaptar a metodologia de datalogging fornecida (rotinas *datalog* e *datalog_sr*).

O código de empacotamento foi reestruturado para que a word de registo (%MW94) combinasse o estado das entradas, saídas e variáveis de controlo interno. Concretamente, os dados foram mapeados bit a bit da seguinte forma:

- Bits 0 a 3: Estado das linhas do teclado (*line_a* a *line_d*).
- Bits 4 a 6: Estado das colunas do teclado (*col_1* a *col_3*).
- Bit 7: Estado do alarme (*O_buzzer*).
- Bits 8 a 11: Valor da tecla lida (*current_key*), deslocado usando multiplicação por 256.
- Bits 12 a 15: Posição atual no array de memória (*msg_index*), deslocado 12 bits através da função *SHL*.

Desta forma, a rotina *datalog_sr* regista o ciclo de scan do PLC e o estado combinado destas variáveis sempre que ocorre uma alteração (gravação por mudança de estado). Isto permite garantir que nenhum evento rápido de pressão de tecla é perdido antes da exportação para o MATLAB.

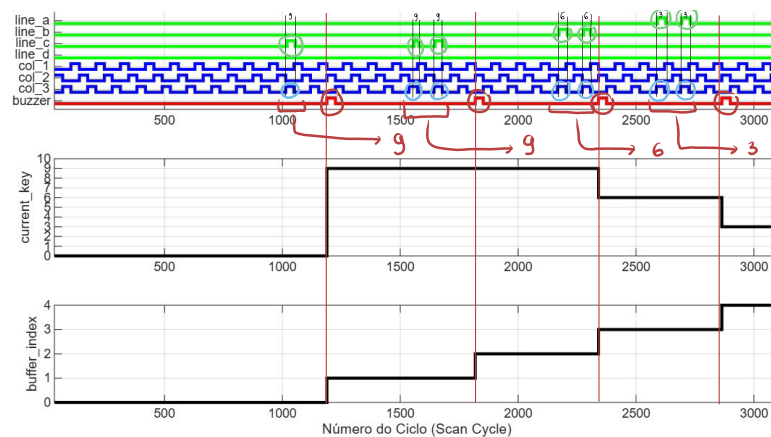


Figura 14: Gráficos temporais dos sinais de entrada, estado da tecla atual e índice do buffer durante a inserção do código 9963.

A figura 14 ilustra o comportamento do programa durante a introdução da sequência pedida no enunciado "9963". Através da análise dos três sub-gráficos, é possível validar o correto funcionamento da lógica implementada e das rotinas de leitura e armazenamento de dados.

Sinais Físicos (gráfico superior): Observa-se o sinal de onda quadrada nas colunas (col_1 a col_3), responsável pela energização contínua do teclado. Quando uma tecla é premida, regista-se a ativação simultânea da linha e da coluna correspondentes. Por exemplo, a deteção dos dois primeiros dígitos '9' ocorre quando a line_c e a col_3 se encontram ativas em simultâneo. De seguida, o '6' é detetado pela interseção da line_b com a col_3, e finalmente o '3' pela line_a com a col_3.

Deteção de key_up: O gráfico demonstra que o sistema aguarda a libertação da tecla para processar a informação, ou seja, para ativar o sinal sonoro e guardar a tecla no array. Ilustra também que este processo não é imediato, ocorrendo apenas após 3 ciclos consecutivos sem deteção de toque (validando a lógica de rejeição de ruído). Quando a tecla é mantida premida durante mais do que um ciclo, são visíveis múltiplos flancos nas linhas, muito próximos uns dos outros. No entanto, a tecla premida é registada apenas uma vez, o que confirma a robustez do sistema e o correto funcionamento em toques longos.

Variáveis de Estado (Gráficos Intermédio e Inferior): Em simultâneo com o pulso do buzzer, a variável current_key atualiza para o valor da tecla validada (passando para 9, mantendo-se no 9 no segundo clique, e descendo depois para 6 e 3). Simultaneamente, o buffer_index é incrementado de forma sequencial, de 0 até 4 (subindo um degrau de cada vez). Isto comprova que as quatro teclas foram corretamente introduzidas e guardadas no vetor de memória, sem perdas de informação ou registos duplicados.

Limite do Buffer e Funcionalidade de *Reset*

Para além da validação de uma sequência normal de teclas, foram realizados testes adicionais para comprovar o comportamento da sub-rotina SR_get_multiple_keys perante condições limite,

nomeadamente a saturação do *buffer* e a ordem de limpeza.

Saturação do Buffer

É de notar que, conforme as especificações originais do sistema, o limite estrutural do buffer é de 10 teclas.

Contudo, exclusivamente para a realização deste teste de saturação, o limite foi temporariamente reduzido para 4. Esta alteração teve como único propósito facilitar o armazenamento de memória e tornar a visualização gráfica do bloqueio mais perceptível, sem comprometer a lógica do programa.

Na Figura 15, é visível o registo temporal de sucessivas pressões na tecla '1'. O gráfico inferior comprova que a variável *buffer_index* incrementa linearmente em cada deteção válida até atingir o limite estipulado de 4.

Após este valor, o índice satura e o sistema ignora as pressões seguintes, comprovando que o mecanismo de proteção contra o transbordo do buffer funciona corretamente.

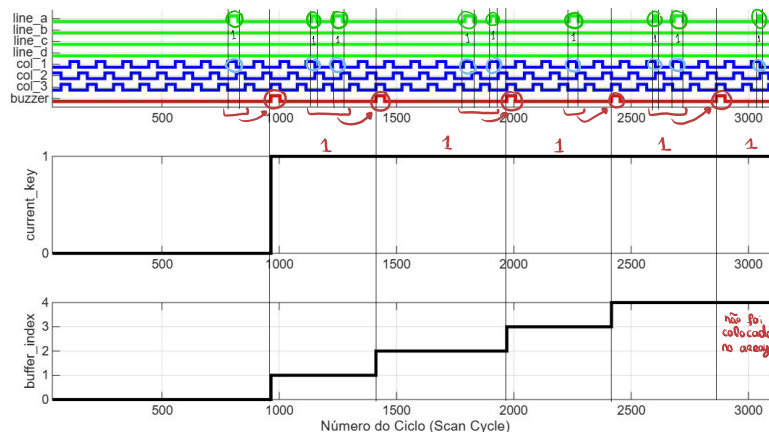


Figura 15: Gráficos temporais demonstrando o limite máximo de armazenamento do buffer (saturação no índice 3).

Funcionalidade de Reset (Flush)

Para demonstrar a funcionalidade de limpeza do vetor de memória, o gráfico da Figura 16 ilustra o comportamento do sistema quando é dada a ordem de reinicialização.

Na primeira fase do gráfico, observa-se a deteção e validação de uma tecla '1' (identificada pela interseção da *line_a* com a *col_1*), que provoca a transição da *current_key* e do *buffer_index* para o valor 1.

Posteriormente, em torno do ciclo de scan 3100 (assinalado com a marcação manual "Reset"), é acionada a rotina de limpeza do sistema (variável *reset_buffer*).

Como se pode verificar nos dois subgráficos inferiores, esta ação força a reposição imediata e simultânea da variável *buffer_index* a zero, limpando logicamente o array de armazenamento e apagando o histórico de teclas inseridas. Em simultâneo, a *current_key* é também reposta a -1 (no gráfico aparece 0, porque só guardamos *current_key* - 1 no datalog).

Este comportamento comprova que o sistema descarta com sucesso as introduções prévias e fica imediatamente pronto para registar uma nova sequência, validando a lógica de reinicialização implementada na rotina *SR_get_multiple_keys*.

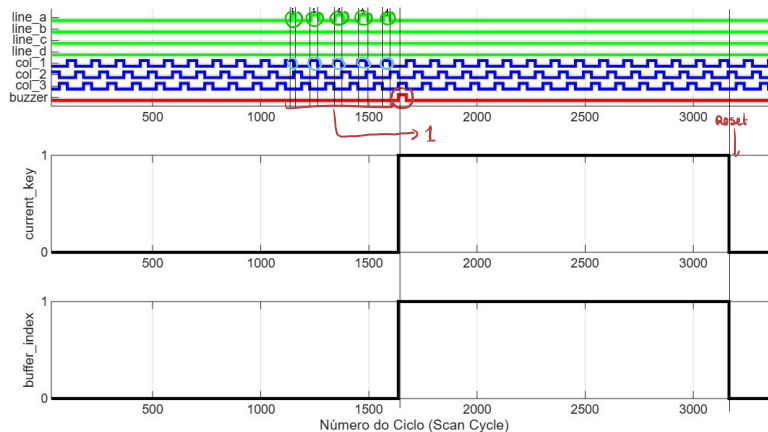


Figura 16: Gráficos temporais ilustrando a limpeza do buffer e o recomeço do índice de armazenamento.

Parte C - Integração da leitura do teclado e dos modos de alarme

C1 - Desenvolvimento da Máquina de Estados

Na Parte C foi implementada uma máquina de estados de nível superior para integrar o teclado matricial com o sistema de alarme. A variável `M_STATE` representa o estado da máquina, enquanto `M_MODE` seleciona a rotina principal executada pelo programa: *OFF*, deteção de presença ou alarme ativo.

Os comandos usados no teclado foram:

Tabela 4: Comandos principais do teclado.

Comando	Função
#1*	Pedido de modo <i>OFF</i> ou início da desativação
#2*	Seleção do modo de deteção de presença
#3*	Pedido de ativação do alarme

Tabela 5: Estados da máquina de estados da Parte C.

Estado	Nome	Descrição
S0	OFF	Estado inicial e seguro do sistema. O alarme encontra-se desligado.
S1	Presença	Modo de deteção de presença.
S2	Espera código de ativação	Estado onde o sistema aguarda a introdução do código 1234 para ativar o alarme.
S3	Atraso de saída	Estado de espera de 30 segundos antes da entrada no modo ativo.
S4	Alarme ativo	Estado em que o sistema se encontra armado e em vigilância.
S5	Aviso de intrusão	Estado de aviso após a deteção de uma possível intrusão.
S6	Buzzer ativo	Estado em que o buzzer do alarme é ativado.
S7	Espera código de desativação	Estado onde o sistema aguarda o código correto para desativar o alarme.
S8	Confirmação OFF	Estado de confirmação da desativação, com emissão de três beeps.
S9	Código errado	Estado associado à introdução de um código incorreto.

A máquina de estados implementada é apresentada na Figura 17. Foram definidos estados adicionais para a validação do código, atraso de saída, aviso de intrusão, ativação do buzzer e confirmação de desativação.

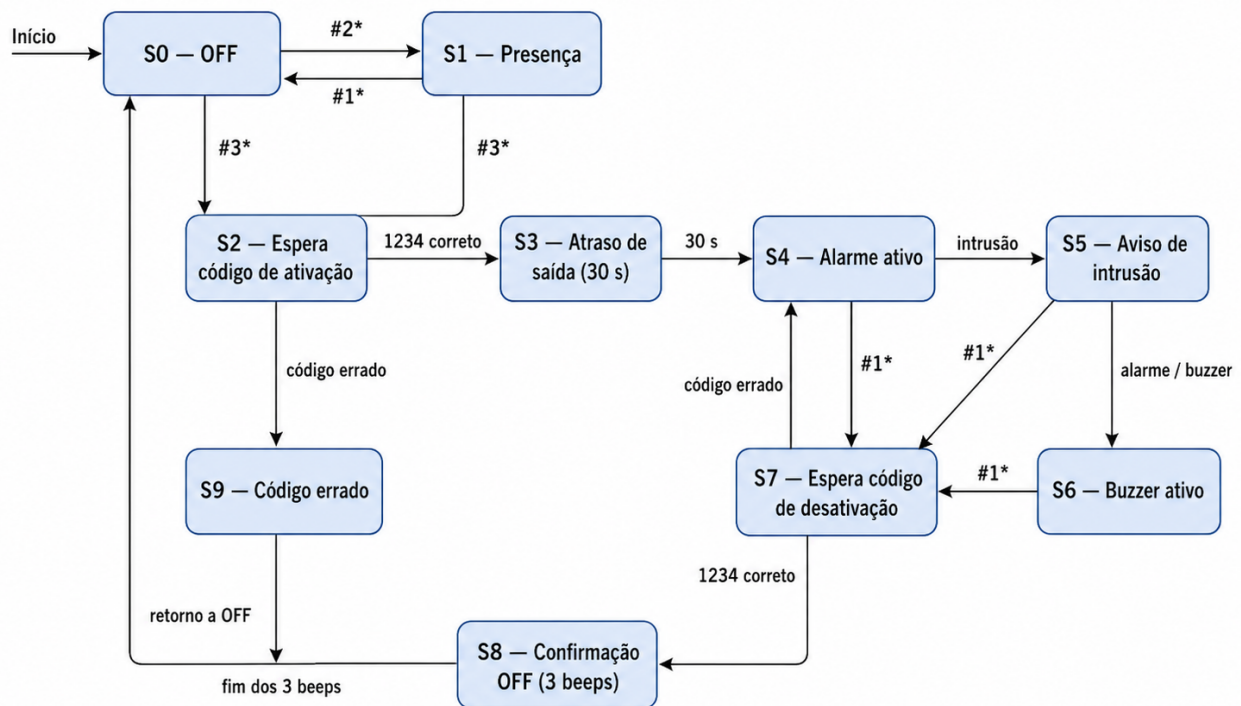


Figura 17: Máquina de estados

O estado inicial é S0, correspondente ao modo *OFF*. O comando #2* coloca o sistema no modo de deteção de presença. O comando #3* inicia a ativação do alarme, mas só após a introdução correta do código guardado em `code[0..3]` e da confirmação com a tecla * é que o sistema passa para o atraso de saída e, depois, para o estado ativo.

Quando o alarme está em atraso de saída, ativo, em aviso de intrusão ou com o buzzer ligado, o comando #1* inicia a desativação. O sistema passa para S7, onde aguarda a introdução do código correto e a confirmação com a tecla *. Se o código for válido, entra em S8, emite três beeps de confirmação e regressa a S0. Se o código estiver errado, regressa ao estado de alarme ativo.

C2 - Programa para integração no PLC

A integração foi implementada numa secção em Structured Text denominada `C.integracao`, executada ciclicamente na tarefa *MAST*. Esta secção coordena a leitura do teclado, a interpretação das sequências introduzidas e a atualização da máquina de estados.

A leitura física do teclado foi mantida nas subrotinas da Parte B. A rotina `SR_get_multiple_keys()` guarda as teclas válidas no vetor `message`, sendo `msg_index` usado para indicar a posição atual do buffer. A secção de integração apenas interpreta este buffer.

A identificação dos comandos foi feita na subrotina `SR_get_comando()`. Quando é reconhecida uma sequência como #1*, #2* ou #3*, a máquina de estados é atualizada e o sinal `reset_buffer` limpa o vetor `message`.

A validação do código é feita por comparação entre `message[0..3]` e `code[0..3]`, sendo a tecla final * usada para confirmar o fim da sequência introduzida. O código inicial usado foi 1234. Se a comparação for correta, a máquina avança para o estado seguinte; se for incorreta, a

transição depende da fase em que o sistema se encontra. Na ativação, um código errado conduz a S9 e depois a S0. Na desativação, um código errado mantém o sistema no modo de alarme ativo.

Foi ainda implementada a alteração dinâmica do código através da subrotina `SR_new_code()`. Quando `I_presence` é ativado, a flag `a_gravar_codigo` permite guardar uma nova sequência de quatro dígitos em `code[0..3]`. Para correção de erros durante a introdução de teclas, a entrada `I_alarm` aciona `reset_buffer`, limpando o buffer e reiniciando `msg_index`.

A variável `M_MODE` foi atualizada em função do estado atual. Os estados S0, S2, S8 e S9 correspondem ao modo *OFF*; S1 corresponde ao modo de presença; e os estados S3, S4, S5, S6 e S7 correspondem ao modo ativo.

C3 - Demonstração do funcionamento

Após a implementação, o programa foi compilado, transferido para o PLC e testado no sistema real. Para validar o funcionamento, foram registadas as variáveis principais através de *data logging*. Os dados foram exportados para um ficheiro `.DTX` e analisados em MATLAB.

O primeiro ensaio validou a sequência completa de ativação e desativação do alarme. Foi introduzida a sequência `#3*1234*`, correspondente ao pedido de ativação seguido do código correto e da confirmação final. Depois do atraso de saída, o sistema entrou no modo ativo. De seguida, foi introduzida a sequência `#1*1234*` para desativar o alarme.

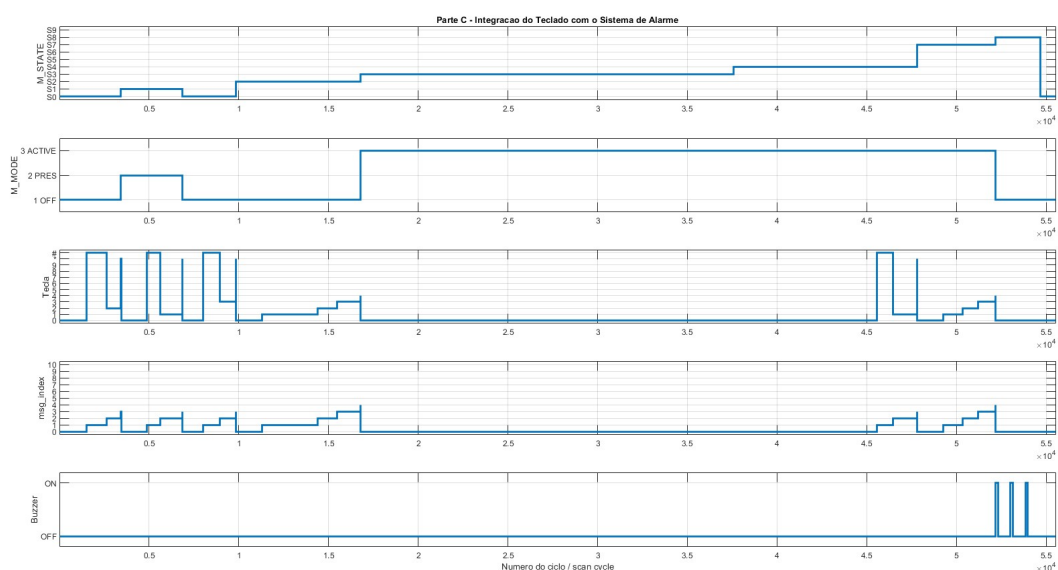


Figura 18: Registo temporal da sequência completa de ativação e desativação do sistema de alarme.

Na Figura 18 observa-se a transição $S0 \rightarrow S2 \rightarrow S3 \rightarrow S4$ durante a ativação. Na desativação, o sistema passa de S4 para S7, valida o código após a confirmação com a tecla *, entra em S8 e regressa a S0. O gráfico de `msg_index` confirma que o buffer é limpo após cada sequência validada, e o gráfico do buzzer mostra a confirmação sonora final.

O segundo ensaio validou a alteração dinâmica do código. Foi gravado o novo código 9963 e, posteriormente, esse código foi usado para ativar o sistema.

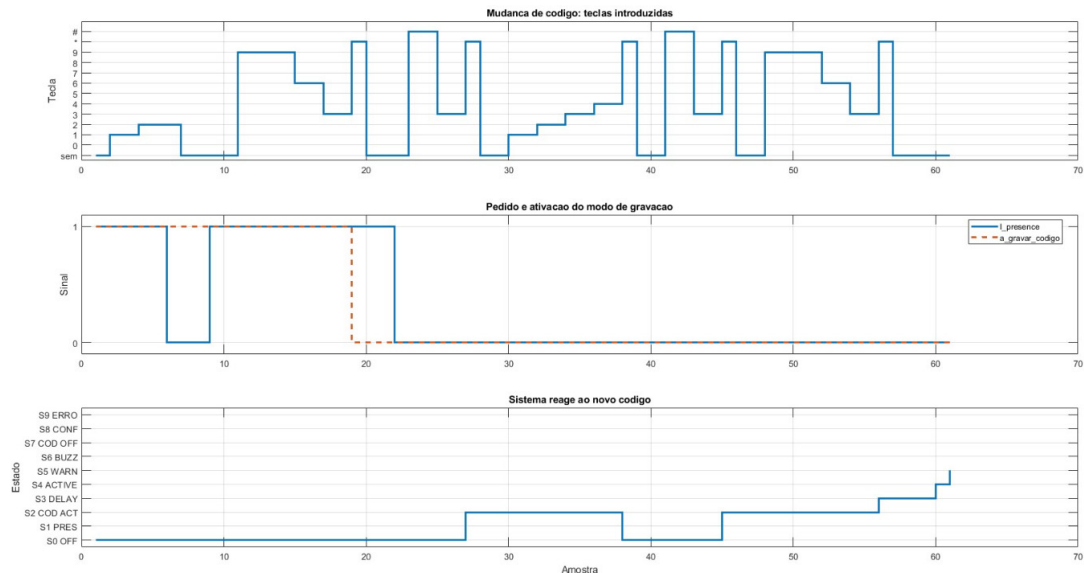


Figura 19: Validação da alteração dinâmica do código de acesso.

Na Figura 19, a ativação de `I_presence` e de `a_gravar_codigo` confirma a entrada no modo de gravação. Após a introdução do novo código, a máquina de estados aceita a sequência e evolui para os estados de ativação do alarme.

O terceiro ensaio validou o reset manual do buffer. Foi introduzida uma sequência parcial e, em seguida, foi ativado `I_alarm` para limpar as teclas guardadas.

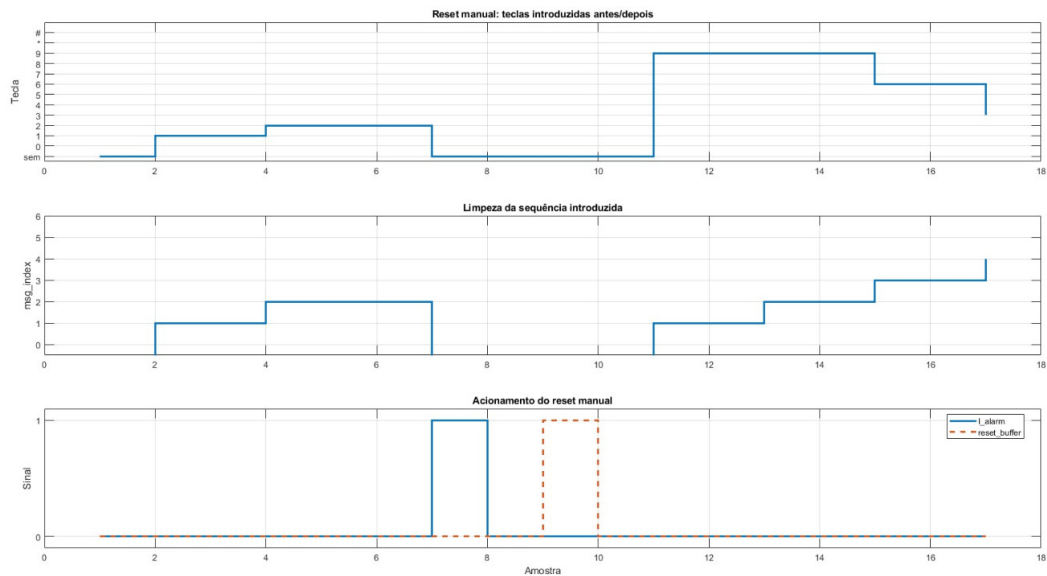


Figura 20: Validação do reset manual do buffer de teclas.

Na Figura 20, o índice `msg_index` aumenta com as teclas introduzidas e regressa a zero após o reset. Este resultado confirma que `reset_buffer` limpa corretamente o vetor `message`, permitindo iniciar uma nova sequência.

Com estes ensaios, foram validadas a ativação e desativação do alarme, a alteração do código de acesso e a limpeza manual do buffer.

C4 - Tempo mínimo para desativar o alarme

Para desativar o alarme a partir do modo ativo, a sequência mínima introduzida no teclado foi:

#1*1234*

Esta sequência tem oito teclas: três para o pedido de desativação, quatro para o código e uma tecla final `*` para confirmar a sequência. O alarme não passa diretamente para *OFF* após `#1*`; primeiro entra em *S7*, onde aguarda a validação do código.

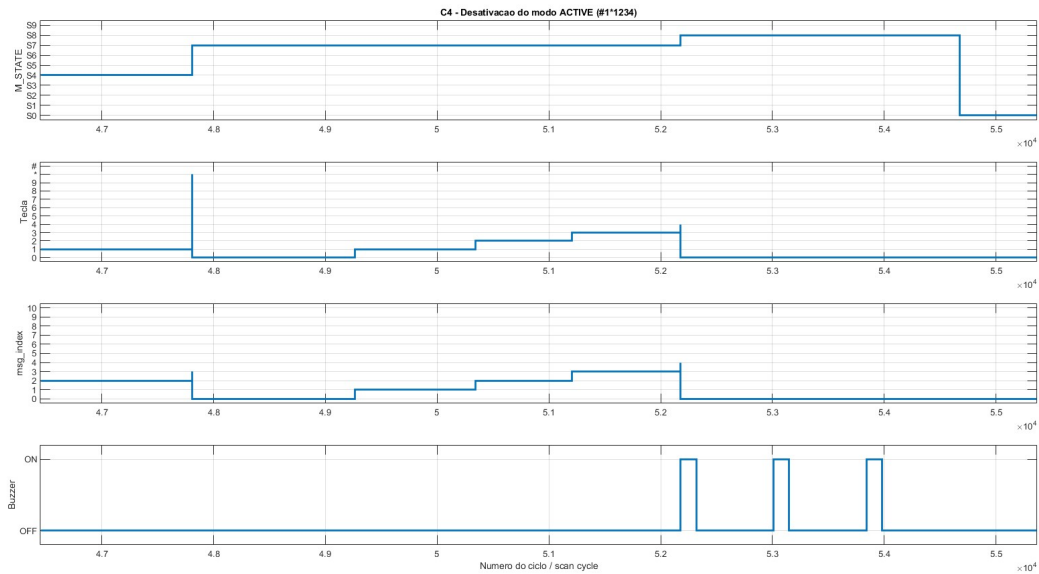


Figura 21: Registo temporal da sequência de desativação do alarme.

Na Figura 21, a máquina encontra-se inicialmente em S4. Após o comando #1*, passa para S7. Depois da introdução correta do código 1234 e da confirmação com a tecla *, transita para S8, onde são emitidos três beeps de confirmação, e regressa finalmente a S0.

No ensaio registado, o primeiro evento associado à desativação ocorre aproximadamente em 4.78×10^4 ciclos de *scan*, e a entrada em S8 ocorre aproximadamente em 5.22×10^4 ciclos. Assim, o tempo mínimo observado até à aceitação da desativação foi de cerca de:

$$5.22 \times 10^4 - 4.78 \times 10^4 \approx 4.4 \times 10^3 \text{ ciclos de } scan.$$

Se for considerado o regresso completo ao estado S0, incluindo a confirmação sonora, o valor observado é de aproximadamente:

$$5.47 \times 10^4 - 4.78 \times 10^4 \approx 6.9 \times 10^3 \text{ ciclos de } scan.$$

Como não foi implementado um *timeout* automático para a introdução do código, não existe um tempo máximo imposto pelo programa. O tempo máximo prático depende apenas do tempo que o utilizador demora a introduzir a sequência completa, desde que o buffer não seja reiniciado manualmente.

C5 - Strings logging

Na alínea C5 tentou-se implementar o registo de mensagens de texto geradas pelo PLC através do terminal virtual `myterminal5`. De acordo com o exemplo fornecido, o terminal lê uma string terminada em zero a partir da zona de memória `%MW190...%MW209`, sendo que cada word de 16 bits armazena dois caracteres ASCII no formato:

$$\%MW = char_1 + 256 \times char_2 \quad (1)$$

Assim, foi acrescentada à secção **C_integracao** uma rotina de escrita de strings associadas às principais transições da máquina de estados. Por exemplo, quando o sistema entra no estado S3 é escrita a mensagem **ACT_1234**, quando passa para S4 é escrita a mensagem **ACTIVE**, durante o pedido de desativação é escrita a mensagem **REQ_OFF**, e durante a confirmação de desativação são escritas as mensagens **OFF_1234** e **OFF**.

Para garantir compatibilidade com o formato esperado pelo terminal, as mensagens foram codificadas em pares de caracteres ASCII. Por exemplo, a string **TEST** pode ser representada por:

```
%MW190 := 84 + 256*69;    (* T E *)  
%MW191 := 83 + 256*84;    (* S T *)  
%MW192 := 0;
```

Foram também copiadas para o projeto as secções auxiliares **mix_IO_ini** e **mix_IO_end**, indicadas no exemplo do terminal, de forma a permitir a comunicação entre o PLC real e o terminal Matlab. A configuração Ethernet foi igualmente verificada, incluindo o endereço IP do PLC e o gateway da rede.

Apesar desta implementação, não foi possível obter a visualização final das mensagens na janela *Logged strings show* do **myterminal5**. O terminal chegou a entrar em modo de execução, mas o campo *PLC string buffer* permaneceu vazio durante os testes. Assim, a validação completa desta alínea não foi concluída experimentalmente.

C6 - Modificações ao data logger para gerar plots em tempo real dos sinais do PLC

Para permitir a criação de gráficos em tempo real num computador em rede, o *data logger* poderia ser adaptado para disponibilizar continuamente os dados do PLC através de comunicação Modbus/TCP. Em vez de exportar manualmente um ficheiro **.DTX**, o PLC escreveria os eventos relevantes numa zona de memória acessível pelo computador.

A estratégia mais adequada seria manter o registo apenas por mudança de estado. Assim, sempre que uma variável importante se alterasse, como entradas, saídas, **M_STATE**, **M_MODE**, tecla atual ou buzzer, o PLC guardaria no buffer um par de valores: o número do ciclo e a word com o estado empacotado.

Para garantir que cada entrada era lida apenas uma vez, o buffer poderia funcionar como uma fila circular. O PLC teria um índice de escrita e o computador teria um índice de leitura. O computador leria apenas os eventos novos, isto é, aqueles que ainda não tinham sido processados. Depois de ler cada entrada, atualizaria o seu índice de leitura.

Desta forma, seria possível atualizar gráficos em MATLAB em tempo real, evitando tanto a repetição de eventos como a perda de informação. O PLC funcionaria como produtor dos dados e o computador como consumidor, lendo periodicamente os novos eventos disponíveis.

C7 - Possíveis melhorias

Como melhoria futura, o sistema de alarme desenvolvido poderia ser expandido para uma solução mais próxima de um sistema industrial real, integrando monitorização remota, interface gráfica e registo histórico de eventos. Atualmente, o sistema funciona localmente através do teclado, LEDs e buzzer, mas poderia ser complementado com uma interface HMI ou SCADA que permitisse visualizar o estado do alarme, consultar eventos anteriores e acompanhar em tempo real as entradas e saídas do PLC.

Uma primeira melhoria seria a implementação de uma interface remota ligada ao PLC por Ethernet/Modbus TCP. Esta interface poderia apresentar o estado atual da máquina de estados, o modo de funcionamento, as últimas teclas introduzidas, o estado dos sensores e o estado das saídas. Desta forma, o operador teria uma visão mais clara do funcionamento do sistema sem depender apenas dos LEDs físicos da consola.

Outra melhoria relevante seria a criação de um histórico de eventos. Sempre que o alarme fosse ativado, desativado, disparado ou sempre que fosse introduzido um código errado, o sistema poderia guardar a data, hora e tipo de evento. Este registo permitiria analisar tentativas de intrusão, erros de utilização e padrões de funcionamento do sistema.

Do ponto de vista da segurança, o sistema poderia também ser melhorado através da alteração dinâmica do código de ativação/desativação, da limitação do número de tentativas erradas e da introdução de bloqueios temporários após várias tentativas falhadas. Estas funcionalidades tornariam o sistema mais robusto e mais próximo de uma aplicação real de controlo de acessos.

Por fim, o sistema poderia ser integrado com notificações remotas, por exemplo através de envio de mensagens para um computador ou aplicação externa sempre que fosse detetada uma intrusão. Esta evolução aproximaria o projeto de temas atuais de automação industrial, como monitorização remota, sistemas ciberfísicos, diagnóstico em tempo real e integração entre PLCs e plataformas digitais.

Assim, o trabalho desenvolvido neste laboratório constitui uma base funcional para um sistema de alarme programável, podendo ser evoluído para uma solução mais completa, segura e conectada.

Conclusões

O presente trabalho permitiu o desenvolvimento e a integração de um sistema de alarme para deteção de intrusão, utilizando um PLC (modelo TSX P57) e o ambiente de desenvolvimento Unity Pro. Através de uma abordagem modular dividida em três fases distintas, foi possível construir progressivamente um sistema de automação robusto e funcional.

Na Parte A, validou-se o funcionamento elementar dos componentes de hardware e das saídas digitais (LEDs e buzzer), implementando os três modos base do alarme: OFF, Deteção de Presença e Ativo. A construção de sub-rotinas em Structured Text e a utilização de temporiza-

dores garantiram uma lógica de controlo fiável, cujo comportamento temporal foi devidamente verificado através da recolha de dados via data logging para o MATLAB.

A Parte B introduziu a complexidade da interação do utilizador através de um teclado matricial 4x3. O desenvolvimento das sub-rotinas de deteção e identificação (SR_get_key) e de armazenamento (SR_get_multiple_keys) assegurou a leitura precisa e sequencial das teclas. A implementação revelou-se eficaz a lidar com problemas típicos de hardware, aplicando mecanismos de rejeição de toques múltiplos simultâneos, deteção de libertação da tecla (key-up) e a correta gestão e reinicialização do buffer de memória.

O culminar do projeto concretizou-se na Parte C, onde se desenhou e implementou uma máquina de estados de nível superior que unificou o controlo do alarme e a leitura do teclado. Esta arquitetura permitiu que as transições de estado passassem a ser comandadas exclusivamente por sequências de teclas específicas (1*, 2* e 3*) e ficassem protegidas por um código de acesso dinâmico com 4 dígitos. Embora não tenha sido possível integrar a funcionalidade de registo de strings para o terminal virtual, as lógicas vitais de segurança, incluindo os tempos de atraso de saída, os ciclos de aviso de intrusão e a capacidade de redefinir o código em tempo real foram testadas e totalmente validadas.

Em suma, os objetivos propostos para este laboratório foram amplamente alcançados. O projeto consolidou competências práticas cruciais em programação de PLCs, mapeamento de memória, planeamento de máquinas de estados e análise de diagnóstico via registo de dados. O sistema final atua como uma fundação sólida, demonstrando um elevado potencial de escalabilidade para, no futuro, evoluir de forma contínua para um ambiente de nível industrial real através da adoção de interfaces SCADA, registo remoto de eventos via Modbus/TCP e gestão de utilizadores.